

CURSO 2: ALGORITMOS AVANZADOS Y ESTRUCTURAS DE DATOS

CLASE 02 • NIVEL 2 - INTERMEDIO

Clase 02: Pilas (Stacks) y Colas (Queues) con `collections.deque`

«Pilas LIFO como Platos Apilados y Colas FIFO como la Fila del Supermercado»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 02** del **Curso 2: Algoritmos Avanzados y Estructuras de Datos**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

Objetivos de la Sesión

Competencia Conceptual: Comprender los principios LIFO (Last In, First Out) y FIFO (First In, First Out).

Competencia Práctica: Implementar pilas y colas eficientes en $O(1)$ usando `collections.deque` en Python.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 02: Pilas (Stacks) y Colas (Queues) con collections.deque

Las pilas y colas son estructuras de datos lineales que restringen la inserción y extracción según una disciplina estricta.

☀ Metáfora Central: Pilas LIFO como Platos Apilados y Colas FIFO como la Fila del Supermercado

Una pila es como una torre de platos (el último que pones es el primero que lavas); una cola es la fila del banco (el primero en llegar es el primero en ser atendido).

Principios Teóricos y Modelo Mental

Una lista de Python como cola es ineficiente: `lista.pop(0)` es $O(n)$ porque desplaza todos los elementos en memoria.

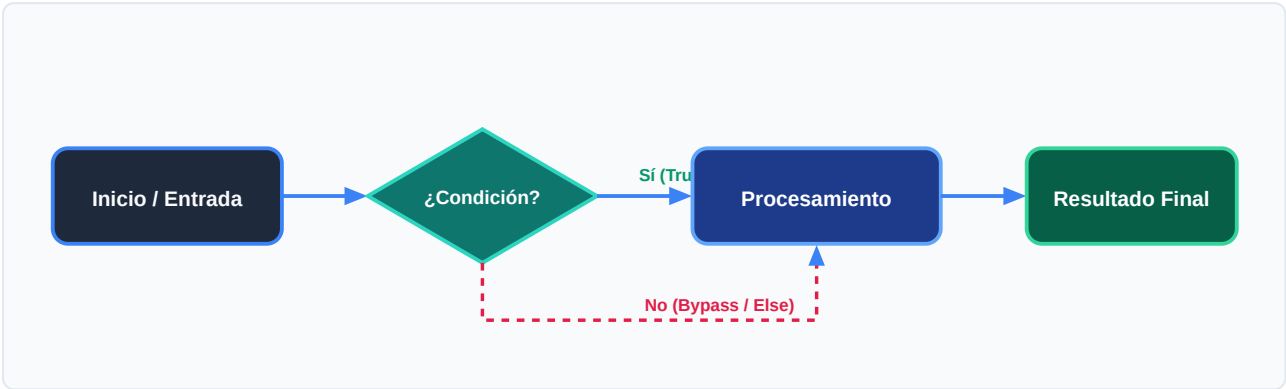
`collections.deque` implementa una lista doblemente enlazada con inserción y extracción $O(1)$ en ambos extremos.

⚡ Regla de Oro en Python

Nunca uses `lista.pop(0)` para colas; usa siempre `collections.deque().popleft()`.

Arquitectura y Movimiento de Datos en Memoria

Mecanismo de inserción y extracción LIFO (Stack) vs FIFO (Queue).



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Inserción de elementos (append / push).	Elemento en el extremo derecho.
2. Evaluación	Operación de consulta del tope (peek).	Lectura sin extracción.
3. Transformación	Extracción LIFO (pop) o FIFO (popleft).	Puntero de nodo actualizado en O(1).
4. Retorno / Salida	Verificación de estructura vacía.	Longitud 0 confirmada.

Visualización Mental

Para deshacer cambios (Ctrl+Z) usa una Pila; para procesar mensajes de cola usa una Cola.

Código de Demostración en Python 3.10+

Validador de paréntesis con pila y procesamiento de cola FIFO:

main.py (Python 3.10+)

PEP 8 Compliant

```
from collections import deque\n\n# 1. Pila (Stack LIFO)\ndef balanceado(expr: str) -> bool:\n    pila = []\n    mapa = {"(": ")", "{": "}", "[": "]"}\n    for char in expr:\n        if char in\n            mapa.values(): pila.append(char)\n        elif char in mapa:\n            if not pila or\n                pila.pop() != mapa[char]: return False\n    return len(pila) == 0\n\n# 2. Cola (Queue\nFIFO)\ncola = deque(["Ticket 1", "Ticket 2", "Ticket 3"])\ncola.append("Ticket\n4")\nprint("Atendido:", cola.popleft()) # Ticket 1\nprint("Es valido:", balanceado("{[()]})")
```

Análisis de Ingeniería del Código

Uso de lista nativa como Pila LIFO y deque para Colas FIFO con rendimiento $O(1)$.



Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Antipatrón de rendimiento en colas.

⚠ Gotcha Frecuente (Trampa de Principiante)

Usar `list.pop(0)` en colas con miles de elementos colapsa el rendimiento de la CPU.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
cola = []
cola.append(x)
primero = cola.pop(0) # ❌ O(n) movimiento de bloques en memoria
```

✅ Patrón Correcto

```
from collections import deque
cola = deque()
cola.append(x)
primero = cola.popleft() # ✅ O(1) instantáneo
```



Consejo de Resiliencia en Producción

`deque` también permite definir `maxlen` para crear buffers circulares de tamaño fijo.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 02: Pilas (Stacks) y Colas (Queues) con collections.deque**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Pilas LIFO como Platos Apilados y Colas FIFO como la Fila del Supermercado
2. Regla de Oro	Nunca uses <code>lista.pop(0)</code> para colas; usa siempre <code>collections.deque().popleft()</code> .
3. Gotcha Crítico	Usar <code>list.pop(0)</code> en colas con miles de elementos colapsa el rendimiento de la CPU.
4. Buenas Prácticas	<code>deque</code> también permite definir <code>maxlen</code> para crear buffers circulares de tamaño fijo.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

Desafío Práctico para el Estudiante

Reto de la Semana

Implementa un historial de navegación web con funciones `ir_a(url)`, `atras()` y `adelante()` usando dos pilas.

Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.